Graph-Based Vulnerability Detection for Software Supply Chain Security: A Comprehensive Survey of Techniques, Datasets, and Emerging Research Challenges

Eman Al-zalab, Sana'a University, Yemen ,eman.alzalab@su.edu.ye, **Research Supervisor**: Dr. AHMAD ALSHALABI

***Abstract*— Abstract—Software vulnerabilities remain a major threat to modern software systems and digital infrastructures. Numerous detection techniques have been proposed, ranging from traditional program analysis methods such as static analysis, dynamic analysis, and fuzz testing to recent machine learning–based approaches. Among these, graph-based learning methods have shown strong capability in modeling structural and semantic relationships within source code.**

This survey analyzes recent advances in vulnerability detection with a focus on graph-based approaches. It examines representative frameworks, widely used datasets, and emerging domains including software supply chain security and blockchain-based smart contracts. Through comparative analysis, the survey highlights key strengths and limitations of existing methods and identifies important research challenges related to scalability, interpretability, and dataset quality.

***Index Terms*—Software vulnerabilities , vulnerability detection, graph neural networks, code property graph, software supply chain security, smart contract security.**

I. Introduction

Modern software systems face escalating cybersecurity challenges, where vulnerabilities constitute one of the most critical sources of exploitation and large-scale attacks [4],[6],[15]. As software complexity increases and integrates with diverse operational environments, vulnerabilities are no longer isolated technical flaws but strategic weaknesses that threaten institutional stability and digital infrastructure [16],[17]. Addressing these challenges requires detection methods that combine accuracy, scalability, and interpretability, ensuring that vulnerabilities are not only identified but also explained in ways that support practical remediation [4],[18]. Traditional detection methods such as static and dynamic analysis have provided valuable insights, yet they frequently suffer from high false-positive rates and limited scalability when applied to complex, real-world systems [19],[6]. Similarly, fuzz testing has demonstrated strong capability in discovering memory-related vulnerabilities through automated input generation; however, its dependence on effective input mutation and coverage-guided strategies often limits its ability to expose deeper logical vulnerabilities within complex program states [20],[21]. Recent advances in graph-based learning have improved detection accuracy by modeling structural and semantic relationships within code [5],[7],[8].However, these approaches remain computationally intensive and often lack interpretability, which hinders their adoption in practical development environments [5],[18]. Taken together, these limitations underscore the need for approaches that bridge the gap between theoretical advances and practical applicability.

II. Background and Definitions

Software vulnerabilities represent weaknesses in code that attackers exploit to compromise system security. A major category arises from memory-related programming errors, particularly in low-level languages such as C and C++. These weaknesses often stem from unsafe pointer manipulation and improper memory management in large-scale systems such as the Linux Kernel, FFmpeg, QEMU, and Wireshark. Vulnerability detection frameworks like Devign analyze functions extracted from these systems to evaluate automated detection models [5]. Despite their prevalence, memory-related vulnerabilities remain difficult to detect reliably, as traditional tools often fail to capture the subtle semantic dependencies that lead to exploitation [22].

Another critical class involves memory-safety vulnerabilities, especially buffer overflows, where programs write data beyond allocated boundaries. Such flaws may overwrite adjacent memory regions and allow attackers to manipulate execution flow. Studies such as VulPathFinder analyze these vulnerabilities using datasets like the Software Assurance Reference Dataset (SARD), which contains numerous memory-related weaknesses including CWE-121 and CWE-126 [7]. While datasets such as SARD provide valuable benchmarks, they may not fully capture the diversity of vulnerabilities encountered in modern, large-scale systems [4].

Beyond memory issues, input validation vulnerabilities occur when applications fail to properly validate external inputs, enabling injection attacks and manipulation of program

behavior. Frameworks such as VISION evaluate detection models using datasets like PrimeVul, which contain instances

of improper input validation (CWE-20) [23]. However, input validation vulnerabilities are highly context-dependent, and existing datasets often lack the breadth needed to generalize across diverse application domains [24],[25].

Large-scale datasets derived from security reports, including BigVul and CVEfixes, further capture diverse vulnerability categories observed in real-world systems. Studies such as COCA and CFExplainer evaluate detection models using these datasets, which collect vulnerable code instances derived from the Common Vulnerabilities and Exposures (CVE) database [14], [18]. While these datasets provide scale and realism, they may also introduce noise and labeling inconsistencies that complicate model training and evaluation [25].

In addition, software supply chain vulnerabilities have emerged as a pressing concern, where weaknesses propagate through third-party libraries and complex dependency structures. Studies such as VDGraph investigate these risks using data from CVE and GitHub Security Advisories (GHSA) [10]. Yet, supply chain vulnerabilities remain underexplored compared to program-level flaws, leaving significant gaps in detection coverage [13].

In blockchain environments, vulnerabilities often stem from logical flaws in smart contract execution, including reentrancy attacks, integer overflows, timestamp dependence, unchecked external calls, denial-of-service vulnerabilities, and access control weaknesses. Graph-based detection frameworks such as the approach proposed by Xu et al. analyze large datasets of smart contracts to learn vulnerability patterns from both bytecode and source-level representations [11]. Despite these advances, smart contract vulnerabilities evolve rapidly, and existing detection frameworks struggle to keep pace with emerging attack vectors [26].

Finally, vulnerabilities may also arise from socio-technical weaknesses in open-source ecosystems, where attackers exploit governance structures and maintainer trust rather than code itself. A notable example is the XZ Utils supply chain attack (CVE-2024-3094), analyzed by Przymus and Durieux, where an attacker infiltrated the development community and injected a malicious backdoor into the release process [13]. Such incidents highlight that vulnerability detection must extend beyond technical code analysis to encompass broader ecosystem risks [27].

Research Gap and Contribution

Despite the progress achieved through traditional program analysis and recent graph-based learning approaches, current methods remain constrained by several critical limitations. Traditional techniques such as static, dynamic, and fuzz testing struggle with scalability and semantic complexity, often producing false positives or failing to uncover deeper logical flaws [1]-[3]. Graph-based approaches, while more accurate, are computationally demanding and frequently lack interpretability, limiting their practical adoption in real-world development environments [5][8]. Furthermore, existing datasets, though extensive, often suffer from noise, inconsistencies, or insufficient coverage of emerging vulnerability categories, particularly in supply chain ecosystems and blockchain environments [4][10].

These shortcomings highlight a clear research gap: the need for vulnerability detection frameworks that integrate structural precision, computational efficiency, and interpretability, while maintaining robustness across diverse software ecosystems. This study addresses that gap by advancing graph-based vulnerability detection with enhanced explanatory mechanisms, aiming to deliver models that not only achieve high detection accuracy but also provide actionable insights into the root causes of vulnerabilities. In doing so, it contributes to bridging the divide between theoretical advances and practical applicability, thereby supporting the development of more secure and trustworthy software systems.

III. Survey Of Literature

This section reviews existing studies related to software vulnerability detection and security analysis. The discussion begins with traditional vulnerability detection techniques that rely on program analysis methods, followed by recent graph-based approaches that leverage machine learning and graph neural networks for software security analysis.

A. Traditional Vulnerability Detection

This Prior to the adoption of deep learning techniques, vulnerability detection primarily relied on traditional program analysis techniques such as static analysis, dynamic analysis, and fuzzing [1]–[3]. Static analysis examines source code without executing programs to identify potential vulnerabilities through predefined rules and code patterns [1]. Dynamic analysis techniques analyze program behavior during runtime to detect security flaws that may only appear during program execution [2]. Fuzzing techniques attempt to uncover vulnerabilities by generating large volumes of random or malformed inputs to trigger unexpected program behavior [3]. Although these approaches have been widely used in software security analysis, they rely heavily on manually crafted rules and expert knowledge, limiting their ability to detect complex vulnerabilities in modern large-scale software systems [4]. These limitations led to the development of data-driven representation learning techniques, particularly graph-based approaches capable of capturing structural relationships within software programs [5].

TABLE I
Comparison of Traditional Vulnerability Detection Methods.

Method	Representative Studies	Strengths	Limitations
Static Analysis	Chess & West [1], Yamaguchi et al. [6]	Detects vulnerabilities early without executing the program	High false-positive rate and limited runtime context
Dynamic Analysis	Shoshitaishvili et al. [2]	Observes real program behavior during execution	Limited coverage depending on test scenarios
Fuzz Testing	Sutton et al. [3]	Effective in discovering memory-related vulnerabilities	Highly dependent on input generation strategies

This table summarizes major traditional vulnerability detection techniques, highlighting representative studies, strengths, and key limitations.

B. Graph-Based Vulnerability Detection

Graph-based learning techniques have recently emerged as a promising direction for vulnerability detection, as they enable modeling complex structural relationships within source code that traditional methods fail to capture. By representing programs as graphs that encode syntactic and semantic dependencies, these approaches allow machine learning models to analyze software behavior more effectively.

Early research in graph-based vulnerability detection primarily focused on applying graph neural networks to analyze structural relationships within source code.
Zhou et al. introduced Devign, one of the earliest frameworks to apply Graph Neural Networks (GNNs) for automated vulnerability detection in source code[5]. Their approach addressed the shortcomings of traditional detection techniques that often fail to capture complex semantic dependencies within software systems. By modeling source code as program graphs encoding both syntactic and semantic relationships, the framework enabled structural analysis of software behavior and achieved superior classification of vulnerable functions compared to conventional machine learning and static analysis methods. The evaluation on real-world datasets from open-source projects such as the Linux Kernel, FFmpeg, QEMU, and Wireshark further underscored the practical relevance of the study, establishing Devign as a seminal contribution in the literature on software security analysis. Nevertheless, the framework exhibits notable limitations: it focuses primarily on vulnerability classification without offering explainable localization, its deep graph neural architecture introduces significant computational complexity with over one million trainable parameters, and it does not extend to modeling dependencies across software supply chain environments. Thus, while Devign represents a foundational step toward graph-based vulnerability detection, subsequent research has focused on improving interpretability and vulnerability localization within graph-based security analysis frameworks[5].

To improve vulnerability detection accuracy, subsequent research has explored more expressive graph representations capable of modeling diverse program relationships.Luo et al. introduced a heterogeneous graph neural network-based framework for vulnerability detection designed to address limitations of homogeneous graph representations that treat all program elements uniformly[8]. Their approach addresses shortcomings of traditional vulnerability detection models that often struggle to capture diverse structural relationships within complex software programs. By modeling source code using Inter-Procedural Abstract Graphs (IPAGs) that integrate syntactic, semantic, and inter-procedural dependencies such as function calls and data flows, the framework enables richer structural representation of program behavior. A Heterogeneous Attention Graph Neural Network (HAGNN) is employed to learn multiple dependency patterns across heterogeneous graph components and improve vulnerability classification performance.The framework was evaluated on datasets derived from the National Vulnerability Database (NVD) and the Software Assurance Reference Dataset (SARD), covering more than one hundred vulnerability categories. The experimental results demonstrate strong detection performance, achieving an accuracy of 96.6% for C programs and 97.8% for Java programs. These findings highlight the effectiveness of heterogeneous graph modeling for vulnerability detection in complex software systems. Nevertheless, the framework remains limited to single-program vulnerability analysis and does not explicitly model vulnerability propagation across software supply chain environments, restricting its applicability to modern interconnected software ecosystems. Thus, while heterogeneous graph modeling improves structural vulnerability representation, further research must extend such approaches toward dependency-aware models capable of analyzing vulnerability propagation across software supply chains[8].

Peng et al. introduced ANGEL, a hierarchical and context-aware graph-based framework designed to improve vulnerability detection in large and structurally complex code graphs[9]. Their approach addresses limitations of conventional static analysis and deep learning techniques, which often fail to effectively capture long-range semantic dependencies and suffer from scalability challenges when applied to large and structurally complex software systems. By modeling source code using Code Property Graphs (CPGs) that unify Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Program Dependency Graphs (PDG), the framework enables comprehensive representation of both syntactic and semantic program relationships. ANGEL integrates Graph Neural Networks (GNNs) with Graph Transformer architectures to capture both local structural dependencies and global contextual interactions within software programs. To address redundancy and scalability issues in large code graphs, the framework introduces a hierarchical graph refinement mechanism that compresses graph structures while preserving vulnerability-relevant information.

The framework was evaluated on widely used benchmark datasets including Devign, Reveal, and Big-Vul, which contain vulnerability samples derived from CVE repositories. Experimental evaluation demonstrates that ANGEL outperforms several state-of-the-art approaches, including token-based models such as VulDeePecker and SySeVR and graph-based frameworks such as Devign, IVDetect, DeepWukong, and MAGNET. These findings highlight the effectiveness of hierarchical graph learning for scalable vulnerability detection in large and structurally complex software systems. Nevertheless, the framework primarily focuses on intra-program vulnerability detection and does not explicitly model dependency relationships across software ecosystems or software supply chain environments, limiting its applicability to modern interconnected software systems. Thus, while ANGEL improves scalability and long-range dependency modeling in graph-based vulnerability detection, further research must extend such architectures toward dependency-aware security models capable of analyzing vulnerability propagation across software supply chains[9].

C. *Explainable Graph-Based Vulnerability Detection*

Atashin et al. introduced VulPathFinder, a graph-based framework designed for explainable vulnerability path discovery in open-source software systems[7]. Their approach addresses a critical limitation of conventional vulnerability detection techniques that typically identify vulnerable code segments without providing insight into the underlying causes of security flaws. By modeling source code using Code Property Graphs (CPGs) that integrate control-flow and data-flow dependencies, the framework enables comprehensive structural analysis of program behavior and facilitates the identification of potential sink points responsible for security weaknesses. A Graph Neural Network (GNN) is employed to detect candidate

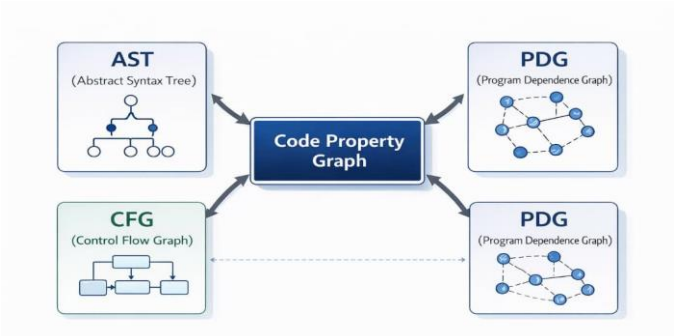
vulnerability sinks, followed by backward program slicing to reconstruct potential vulnerability execution paths and reveal the causal relationships leading to security flaws. The framework was evaluated on the Software Assurance Reference Dataset (SARD), which contains a wide range of vulnerability instances, including memory-related weaknesses such as buffer overflows. Experimental results demonstrate strong vulnerability detection capability, achieving an F1-score of 0.98 and an explanation accuracy of approximately 98%, measured using Triggering Line Coverage metrics. These findings highlight the effectiveness of explainable graph-based analysis for vulnerability localization and demonstrate the potential of graph-based learning techniques for improving transparency in automated vulnerability detection systems.Nevertheless, the framework remains limited to intra-program vulnerability analysis, where vulnerabilities are analyzed within the boundaries of individual software programs. Consequently, the approach does not explicitly consider dependency relationships across software ecosystems or software supply chain environments, which restricts its Thus, while VulPathFinder advances explainable graph-based vulnerability analysis, further research is required to

TABLE 2
COMPARATIVE SUMMARY OF REPRESENTATIVE GRAPH-BASED VULNERABILITY DETECTION FRAMEWORKS.

Framework	Graph Representation	Dataset	Strengths	Limitations
Devign (Zhou et al.) [5]	Program Graph / Code Property Graph encoding syntactic and semantic relationships	Devign dataset derived from Linux Kernel, FFmpeg, QEMU, Wireshark	One of the earliest frameworks applying GNNs for vulnerability detection; effectively captures semantic dependencies in source code; improves classification accuracy over traditional ML and static analysis methods	Lacks explainable vulnerability localization; high computational complexity (>1M parameters); does not model software supply chain dependencies
HAGNN (Luo et al.) [8]	Inter-Procedural Abstract Graph (IPAG) with heterogeneous graph structure	NVD + SARD datasets (100+ vulnerability types)	improves vulnerability classification using attention-based graph learning; high detection accuracy (96.6% for C, 97.8% for Java)	Limited to intra-program analysis; does not model vulnerability propagation across software ecosystems
ANGEL (Peng et al.) [9]	Hierarchical Code Property Graph (AST + CFG + PDG) combined with Graph Transformer	Devign, Reveal, Big-Vul datasets derived from CVE repositories	Captures long-range semantic dependencies; improves scalability for large code graphs; outperforms several state-of-the-art models (Devign, IVDetect, DeepWukong, MAGNET)	Focuses mainly on intra-program vulnerability detection; does not explicitly consider supply chain dependency relationships

Table 2. Comparative summary of representative graph-based vulnerability detection frameworks discussed in this survey, highlighting graph representations, datasets, strengths, and limitations

FIGURE 1. OVERVIEW OF GRAPH REPRESENTATIONS (AST, CFG, PDG, CPG)



This figure illustrates the relationships among graph representations and shows how the Code Property Graph (CPG) integrates AST, CFG, and PDG.

extend such approaches toward dependency-aware security models capable of analyzing vulnerability risks across complex software supply chains.Compared with traditional vulnerability detection techniques that primarily focus on identifying vulnerable code segments, VulPathFinder represents an early attempt to provide explainable vulnerability localization through graph-based program representations. However, subsequent research has explored more advanced explanation mechanisms that integrate causal reasoning and representation learning to further improve both interpretability and robustness in graph-based vulnerability detection models[7].

Cao et al. introduced COCA, a contrastive causal explanation framework designed to enhance the interpretability and robustness of Graph Neural Network (GNN)-based vulnerability detection models. As recent graph-based vulnerability detection frameworks have achieved strong detection performance but remain difficult to interpret [14], their approach addresses a critical limitation of existing deep learning–based vulnerability detection systems, which frequently operate as black-box models and therefore fail to provide precise explanations and structural localization of vulnerability-inducing code patterns within complex program graphs. By representing source code using graph-based program representations, including Code Property Graphs (CPGs) and Program Dependence Graphs (PDGs), the framework captures essential structural relationships such as control-flow and data-flow dependencies that are critical for accurate vulnerability analysis. The proposed architecture consists of two main components: COCATra, which improves the robustness of vulnerability detectors through contrastive representation learning and semantic-preserving code transformations, and COCAExp, an explanation module that employs dual-view causal reasoning by combining factual and counterfactual analysis to identify minimal vulnerability-relevant subgraphs responsible for the model's predictions.The framework was evaluated on several widely used vulnerability detection benchmarks, including Devign, Reveal, BigVul, CrossVul, and CVEfixes, which contain large collections of real-world vulnerable and non-vulnerable code functions. Experimental results demonstrate that COCA improves both vulnerability detection performance and explanation quality when compared with existing explainability approaches such as GNNExplainer, PGExplainer, and CFExplainer. By extracting concise program subgraphs that highlight vulnerability-inducing statements, the framework enables more transparent vulnerability analysis and assists developers in identifying the root causes of software vulnerabilities.Nevertheless, the framework exhibits several limitations. First, the proposed explanation mechanism primarily focuses on intra-program vulnerability analysis, where vulnerabilities are detected within individual software systems, and therefore does not explicitly capture dependency relationships across software ecosystems or software supply chain environments. Second,the framework relies on graph representations derived from individual program functions, which limits its ability to model vulnerability propagation across interdependent software components. In addition, generating causal explanations for large program graphs may introduce additional computational overhead, potentially affecting scalability when applied to large-scale software systems.Thus, while COCA represents an important step toward interpretable graph-based vulnerability detection, future research must advance toward scalable, dependency-aware security models capable of analyzing vulnerability propagation across complex software supply chains.While causal explanation frameworks significantly improve the interpretability of graph-based vulnerability detection models, understanding the structural causes that influence model predictions remains a challenging problem. Consequently, recent studies have explored counterfactual reasoning techniques to provide more precise explanations of vulnerability detection decisions.

Despite the substantial progress achieved by graph neural network (GNN)–based vulnerability detection models, their limited interpretability poses a critical challenge in practical software security analysis. While recent deep learning approaches effectively capture structural and semantic patterns in source code, their predictions often remain opaque, hindering developers from understanding the root causes of detected vulnerabilities. To address this challenge, Chu et al. introduced CFExplainer, one of the first frameworks to formulate vulnerability explanation as a counterfactual reasoning problem within graph-based vulnerability detection models [18].By representing source code as program graphs and leveraging differentiable edge masking techniques, the framework identifies minimal structural perturbations capable of altering the model's prediction outcome, thereby revealing the critical subgraphs responsible for vulnerability detection. This counterfactual explanation mechanism facilitates more precise localization of vulnerability-inducing code patterns and enhances the transparency of graph-based vulnerability detection systems. The framework was evaluated on the widely used Big-Vul dataset, a large-scale vulnerability dataset constructed from real-world CVE reports and open-source software repositories. Experimental results demonstrate that CFExplainer consistently outperforms several state-of-the-art explanation techniques, including GNNExplainer, PGExplainer, SubgraphX, GNN-LRP, DeepLIFT, and Grad-CAM, across multiple vulnerability-oriented and model-oriented evaluation metrics. These findings underscore the effectiveness of counterfactual reasoning in accurately localizing vulnerability-inducing code patterns and significantly improving the interpretability of graph-based vulnerability detection models.Nevertheless, the framework exhibits several important limitations. First, CFExplainer focuses primarily on intra-program vulnerability analysis, where explanations are derived from structural perturbations within individual program graphs. Second, the approach fails to capture dependency relationships across software ecosystems or analyze vulnerability propagation across interconnected software components. Finally, the framework relies on existing vulnerability detection models and focuses primarily on explaining model decisions rather than advancing the

detection capability itself.Thus, while CFExplainer constitutes a significant step toward explainable graph-based vulnerability detection, further research must advance toward scalable and dependency-aware security models capable of analyzing vulnerability propagation across complex software supply chains.Although counterfactual explanation techniques substantially enhance model interpretability, robustness and generalization remain important challenges in graph-based vulnerability detection systems. As a result, recent studies have explored counterfactual data augmentation strategies to improve both model robustness and explanation reliability [18].

Despite the substantial progress achieved by graph neural network (GNN)–based approaches for automated vulnerability detection, these models often suffer from limited robustness and poor generalization due to spurious correlations learned from training data. Such correlations may cause vulnerability detectors to rely on superficial code patterns rather than the true semantic causes of vulnerabilities, thereby limiting their reliability in practical software security analysis. To address this challenge, Egea et al. introduced VISION, a framework designed to improve both the robustness and interpretability of graph-based vulnerability detection models through counterfactual data augmentation [23].The proposed framework generates counterfactual code samples that introduce minimal semantic-preserving modifications to source code, enabling the model to distinguish between superficial correlations and the underlying structural causes of vulnerabilities. By leveraging large language models to generate counterfactual examples and representing source code as Code Property Graphs (CPGs), the framework trains a GNN-based vulnerability detector capable of learning more robust and semantically meaningful representations of vulnerable code patterns. In addition, the framework incorporates an explanation module that provides interpretable insights into the model's predictions, allowing developers to identify the code regions responsible for detected vulnerabilities. Extensive experiments conducted on the PrimeVul dataset, focusing on CWE-20 (Improper Input Validation) vulnerabilities, demonstrate that counterfactual augmentation substantially improves both robustness and predictive performance. In particular, balanced training configurations incorporating counterfactual samples achieve superior accuracy, precision, recall, and F1-score compared to models trained solely on original data. Furthermore, embedding space analysis and attribution consistency evaluations reveal that counterfactual augmentation effectively mitigates spurious correlations and promotes more semantically consistent representations of vulnerable and benign code.Nevertheless, the framework exhibits several notable limitations. First, the evaluation focuses on a single vulnerability category (CWE-20), limiting the assessment of the framework across diverse vulnerability types. Second, the generation of counterfactual samples relies on large language models without formal verification of semantic correctness. Third, the framework operates at the intra-program level and does not model vulnerability propagation across software ecosystems or software supply chain dependencies. Thus, VISION represents an important step toward robust and interpretable graph-based vulnerability detection [23].

D. Domin-Specific Applications

Xu et al. introduced a graph-based framework for smart contract vulnerability detection that leverages Graph Neural Networks (GNNs) to overcome the limitations of rule-based static analyzers and conventional machine learning techniques [11]. Their approach addressed the reliance of traditional methods on handcrafted patterns and limited scalability in multi-vulnerability detection scenarios. By modeling smart contracts as graph representations derived from EVM bytecode and Solidity source code using Control Flow Graphs (CFGs) and Abstract Syntax Graphs (ASGs), the framework enabled structural learning of vulnerability patterns directly from program representations. Evaluation on 25,350 smart contracts covering eight vulnerability types achieved strong detection performance, reaching 95.78% accuracy and 93.80% F1-score, underscoring the effectiveness of graph-based learning for blockchain security analysis. Nevertheless, the framework exhibits notable limitations: it focuses on predefined vulnerability categories and does not model vulnerability propagation or zero-day risks across interconnected software ecosystems [11].

Przymus and Durieux introduced a comprehensive software engineering analysis of the XZ Utils supply chain attack (CVE-2024-3094) in their study "Wolves in the Repository" [13], addressing socio-technical vulnerabilities within open-source development ecosystems. Their study addressed the limitations of traditional vulnerability research that often focuses primarily on code-level flaws while overlooking governance manipulation and long-term social engineering strategies targeting project maintainers. By systematically analyzing Git commit histories, mailing-list discussions, and GitHub activity logs, the study enabled a detailed reconstruction of the multi-year infiltration process through which an attacker gradually established credibility and ultimately injected a malicious backdoor into the XZ Utils release pipeline. This analysis underscored the role of governance manipulation and social dynamics in enabling software supply chain compromises, establishing the attack as a significant example of socio-technical security risks in open-source ecosystems. Nevertheless, the study exhibits notable limitations: it relies on a single case study and publicly available artifacts, which limits the generalizability of the findings and may overlook covert coordination among attackers. Thus, while the work represents an important contribution to software supply chain security research, future studies must advance toward developing scalable mechanisms capable of detecting anomalous behavioral patterns across open-source development communities [13].

Xia et al. introduced VDGraph, a graph-based framework for analyzing vulnerabilities in software supply chains by integrating Software Bill of Materials (SBOM) and Software Composition Analysis (SCA) data into a unified dependency graph model [10]. Their approach addressed the limitations of traditional vulnerability assessment techniques that

analyze dependency information and vulnerability reports independently, often resulting in fragmented security visibility. By modeling software projects as dependency graphs representing components and vulnerability relationships, the framework enabled systematic analysis of vulnerability propagation across transitive dependencies. Evaluation on 21 real-world Java projects using vulnerability data from CVE and GitHub Security Advisories (GHSA) demonstrated that many vulnerabilities originate from deeply nested dependencies rather than direct project components, underscoring the importance of dependency-aware analysis in modern software supply chains. Nevertheless, the framework exhibits notable limitations: it relies on structural graph analysis without incorporating machine learning mechanisms capable of predicting unknown or zero-day vulnerabilities. Thus, while VDGraph represents an important step toward improving vulnerability visibility in software supply chains, future research must advance toward adaptive graph learning models capable of predicting vulnerability propagation risks [10].

Jian et al. introduced a graph-based framework for vulnerability propagation analysis in software supply chains using Graph Neural Networks (GNNs) [12]. Their approach addressed the limitations of traditional vulnerability detection techniques that primarily identify isolated software weaknesses without analyzing propagation risks across dependency networks. By modeling the software supply chain as a dependency graph in which nodes represent software components and edges capture dependency relationships, the framework enabled structural analysis of vulnerability transmission across interconnected software ecosystems. Experimental results demonstrated that the GNN-based approach effectively identifies vulnerability propagation paths and improves risk assessment in large-scale software supply chain environments. Nevertheless, the framework exhibits notable limitations: it focuses on known vulnerability propagation patterns and does not explicitly address zero-day vulnerability prediction in evolving software supply chains [12].

Fig. 2. Research landscape of vulnerability detection chniques. The figure illustrates the relationship between traditional analysis methods, machine learning approaches, and graph-based learning frameworks, as well as their applications in software supply chain security and blockchain-based smart contract analysis. It also highlights key research challenges including dataset quality, interpretability, and scalability.

TABLE 3
DOMAIN-SPECIFIC VULNERABILITY DETECTION APPROACHES

STUDY	Domain	Vulnerability Type	Approach	Dataset / Evaluation	Key Strength	Limitations
Xu et al. [11]	Blockchain / Smart Contracts	Reentrancy, integer overflow, access control flaws	Graph-based learning using GNNs with CFG and ASG representations derived from EVM bytecode and Solidity code	25,350 smart contracts covering eight vulnerability types	Learns structural vulnerability patterns directly from smart contract graphs; high detection performance (95.78% accuracy, 93.80% F1)	Limited to predefined vulnerability types; does not model vulnerability propagation or zero-day threats
Przymus & Durieux [13]	Socio-technical supply chain security	Maintainer compromise and malicious code injection (XZ Utils attack)	Software engineering analysis of repository history, mailing lists, and developer activity	Case study of the XZ Utils attack (CVE-2024-3094)	Reveals socio-technical attack strategies targeting governance and maintainer trust	Single case study; limited generalizability for automated detection
Xia et al. (VDGraph) [10]	*Software Supply Chain*	Dependency vulnerabilities across transitive libraries	Graph-based dependency analysis integrating SBOM and SCA data	21 real-world Java projects using CVE and GHSA vulnerability data	Enables analysis of vulnerability propagation through dependency graphs	Does not incorporate machine learning for predicting unknown vulnerabilities
Jian et al. [12]	*Software Supply Chain*	Vulnerability propagation across dependency networks	Graph Neural Network model for analyzing vulnerability propagation paths	Experimental evaluation on supply chain dependency graphs	Identifies vulnerability propagation paths and improves risk assessment	Focuses on known vulnerability patterns; limited capability for zero-day prediction

TABLE 4
COMPARATIVE SUMMARY OF REPRESENTATIVE GRAPH-BASED VULNERABILITY DETECTION FRAMEWORKS

Framework	Graph Representation	Dataset	Key Strength	Limitations
Devign [5]	Code Property Graph (AST + CFG + PDG)	Devign dataset (Linux, FFmpeg, QEMU, Wireshark)	Captures semantic relationships in source code using GNNs	High computational complexity and limited explainability
VulPathFinder [7]	Code Property Graph (CPG)	SARD dataset	Provides explainable vulnerability localization through vulnerability path analysis	Limited to intra-program vulnerability detection
HAGNN [8]	Inter-Procedural Abstract Graph (IPAG)	NVD, SARD	Models heterogeneous program relationships including function calls and data flows	Does not consider software ecosystem dependencies
ANGEL [9]	Hierarchical CPG + Graph Transformer	Devign, Reveal, Big-Vul	Improves scalability for large code graphs using hierarchical graph refinement	Focuses mainly on single-program vulnerability detection
VDGraph [10]	Software Dependency Graph	Java projects with CVE & GHSA	Enables vulnerability propagation analysis in software supply chains	Lacks predictive capability for unknown vulnerabilities
Jian et al. [12]	Dependency Graph + GNN	Supply chain vulnerability datasets	Models vulnerability propagation across dependency networks	Focuses mainly on known vulnerabilities
Xu et al. [11]	CFG + Abstract Syntax Graph (ASG)	Smart contract dataset (25k contracts)	Effective detection of blockchain vulnerabilities using graph learning	Limited coverage of evolving attack patterns

Table 4.1. Comparative summary of representative graph-based vulnerability detection frameworks discussed in this survey, highlighting graph representations, datasets, strengths, and limitations.

IV. COMPARATIVE ANALYSIS AND DISCUSSION

This section presents a comparative analysis of the vulnerability detection techniques discussed throughout this survey, highlighting their strengths, limitations, and emerging research directions. Table 4.1 provides a comparative overview of representative vulnerability detection frameworks in terms of graph representations, datasets, strengths, and limitations. The reviewed literature reveals a clear transition from traditional program analysis techniques toward data-driven and graph-based learning frameworks capable of modeling complex structural and semantic relationships within modern software systems. This evolution reflects the growing demand for scalable and intelligent vulnerability detection mechanisms capable of addressing the increasing complexity of contemporary software ecosystems.

Traditional vulnerability detection techniques such as static analysis, dynamic analysis, and fuzz testing remain foundational approaches in software security analysis. Static analysis enables early vulnerability discovery without executing programs, allowing developers to detect potential weaknesses during early stages of software development. Dynamic analysis complements this capability by observing program behavior during runtime, enabling the identification of vulnerabilities that manifest only during execution. Fuzz testing further enhances vulnerability discovery by generating large volumes of random or malformed inputs designed to trigger unexpected program behavior and expose memory-related flaws. Despite their effectiveness, these techniques often struggle with scalability and semantic complexity when applied to modern large-scale software systems. In particular, static analysis tools frequently produce high false-positive rates due to limited contextual understanding of program semantics, while dynamic analysis and fuzz testing remain constrained by execution coverage and input generation strategies.

To address these limitations, machine learning–based vulnerability detection approaches have been proposed. Early learning-based methods typically rely on token-level representations of source code or manually engineered features. Although such approaches improve automation in vulnerability detection, they often fail to capture deeper structural relationships between program components. Consequently, these methods may struggle to identify complex vulnerability patterns that arise from interactions among multiple program structures.

Graph-based vulnerability detection approaches represent a major advancement in this domain by modeling source code as structured graphs that capture relationships between program elements. Graph representations such as Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Program Dependence Graphs (PDG) allow explicit modeling of syntactic and semantic dependencies within software programs. These representations can be further unified into Code Property Graphs (CPGs), enabling comprehensive program analysis through integrated structural representations. When combined with Graph Neural Networks (GNNs), these representations enable learning models capable of capturing hierarchical semantic relationships within code. Frameworks such as Devign [5] and ANGEL [9] demonstrate the effectiveness of graph-based learning approaches in improving vulnerability detection performance through structural

program analysis.

Despite these advantages, graph-based vulnerability detection frameworks introduce several challenges. One major limitation involves computational scalability, as constructing and processing large program graphs can require substantial computational resources. Additionally, many existing frameworks focus primarily on vulnerability detection within individual programs and do not explicitly consider dependency relationships across software ecosystems. This limitation is particularly critical in modern software supply chains, where vulnerabilities may propagate through complex dependency networks and third-party components. Approaches such as VDGraph [10] and the vulnerability propagation framework proposed by Jian et al. [12] attempt to address this limitation by modeling software dependencies using graph-based representations that enable vulnerability propagation analysis across software ecosystems.

Another significant challenge involves the interpretability of graph-based vulnerability detection models. While deep graph learning models often achieve high detection accuracy, many frameworks provide limited explanations regarding why specific code segments are classified as vulnerable. Although approaches such as VulPathFinder [7] attempt to improve interpretability through vulnerability path analysis, explainable vulnerability detection remains an underdeveloped area of research.

Dataset quality also plays a critical role in the effectiveness of vulnerability detection techniques. Benchmark datasets such as SARD, BigVul, and CVE-based collections [4] provide valuable resources for evaluating vulnerability detection models. However, these datasets often contain noisy samples, inconsistent labeling, and limited representation of emerging vulnerability categories. Consequently, models trained on such datasets may exhibit reduced generalization capability when applied to real-world software environments.

In addition to these challenges, vulnerability detection research is expanding beyond traditional program-level vulnerabilities. Emerging domains such as software supply chain security and blockchain-based applications introduce new classes of vulnerabilities that require specialized detection techniques. In software supply chains, vulnerabilities may propagate through complex dependency relationships among interconnected software components. In blockchain environments, vulnerabilities frequently arise from logical flaws within smart contract execution mechanisms.

Overall, the comparative analysis reveals several important research trends. First, graph-based learning frameworks consistently outperform feature-based methods in modeling complex semantic relationships within source code. Second, most existing vulnerability detection approaches remain limited to intra-program analysis and rarely consider vulnerability propagation across software ecosystems. Third, explainable vulnerability detection remains an open research challenge despite advances in deep learning–based approaches. Finally, emerging domains such as software supply chain security and blockchain systems are receiving increasing research attention due to the growing complexity of modern software infrastructures.

These observations highlight several important research challenges that must be addressed in future work. In particular, future vulnerability detection frameworks should focus on improving scalability for large program graphs, enhancing interpretability to support practical vulnerability remediation, and developing higher-quality datasets capable of representing diverse vulnerability categories across evolving software ecosystems. Addressing these challenges will be essential for building robust and trustworthy vulnerability detection systems capable of supporting secure software development in increasingly interconnected computing environments.

FIGURE 3 ILLUSTRATES THE DISTRIBUTION OF RESEARCH FOCUS ACROSS DIFFERENT VULNERABILITY CATEGORIES CONSIDERED IN THIS SURVEY.

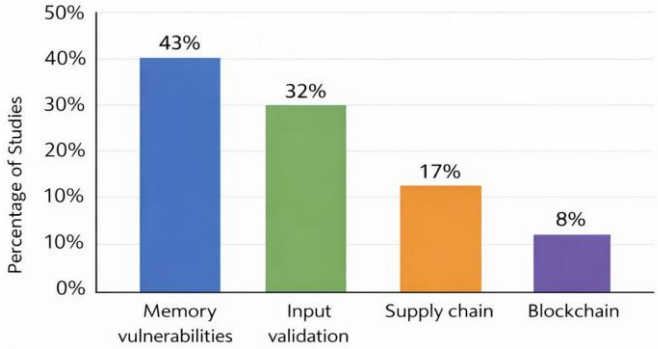


Fig. 3. DISTRIBUTION OF RESEARCH FOCUS ACROSS VULNERABILITY CATEGORIES BASED ON THE REPRESENTATIVE STUDIES ANALYZED IN THIS SURVEY.

Figure 3 illustrates the distribution of research focus across the vulnerability categories examined in this survey based on the representative studies analyzed in Sections 3.1–3.3. The reviewed literature reveals a comparable concentration on memory-related vulnerabilities, input validation issues, and software supply chain security, each representing a substantial portion of the analyzed studies. In contrast, blockchain-related vulnerabilities receive noticeably less attention within the current body of research.

This distribution reflects the historical emphasis of software security research on traditional program-level vulnerabilities, particularly memory safety errors and input validation flaws, which have long been recognized as major sources of software exploitation. At the same time, the growing presence of studies focusing on software supply chain security highlights the increasing awareness of dependency-based risks in modern software ecosystems.

Nevertheless, the relatively limited number of studies addressing blockchain security indicates an emerging research direction that remains insufficiently explored. Given the rapid expansion of decentralized applications and smart contract platforms, this observation suggests the need for more

advanced vulnerability detection approaches specifically designed for blockchain-based software systems.

V. Conclusion

A conclusion Software vulnerabilities remain a critical threat to modern software systems and digital infrastructures. This survey reviewed major vulnerability detection techniques, ranging from traditional program analysis methods to recent graph-based learning approaches. While static analysis, dynamic analysis, and fuzz testing remain valuable for vulnerability discovery, they often suffer from scalability limitations and high false-positive rates in complex software systems. Graph-based approaches have significantly improved detection capability by modeling structural and semantic relationships within source code using representations such as AST, CFG, and CPG combined with graph neural networks. However, these approaches still face challenges related to computational complexity, interpretability, and dataset quality. Future research should focus on developing scalable, explainable, and ecosystem-aware detection frameworks capable of addressing vulnerabilities across modern software environments, including software supply chains and blockchain systems.

References

[1] B. Chess and J. West, Secure Programming with Static Analysis. Boston, MA, USA: Addison-Wesley Professional, 2007.

[2] Y. Shoshitaishvili et al., "State of the art: Automated dynamic vulnerability discovery," in Proc. IEEE Symp. Security Privacy (SP), San Jose, CA, USA, 2016.

[3] M. Sutton, A. Greene, and P. Amini, Fuzzing: Brute Force Vulnerability Discovery. Boston, MA, USA: Addison-Wesley Professional, 2007.

[4] S. Chakraborty et al., "Deep learning–based vulnerability detection: Are we there yet?" IEEE Trans. Softw. Eng., 2020

[5] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[6] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in Proc. IEEE Symp. Security Privacy (SP), 2014.

[7] Atashin et al., "Learning to locate: GNN-powered vulnerability path discovery in open source code," 2023.

[8] L. Luo et al., "Detecting code vulnerabilities with heterogeneous graph neural networks," 2023.

[9] Peng et al., "Keep it simple: Towards accurate vulnerability detection for large code graphs (ANGEL)," 2024.

[10] X. Xia et al., "VDGraph: Graph-based software supply chain vulnerability analysis," 2024.

[11] Y. Xu et al., "Enhanced smart contract vulnerability detection via graph neural networks," IEEE Access, 2025.

[12] Jian et al., "Construction of graph neural network vulnerability propagation identification mechanism for software supply chain," 2025.

[13] P. Przymus and T. Durieux, "Wolves in the repository: A software engineering analysis of the XZ utils supply chain attack," 2024.

[14] C. Cao et al., "COCA: Contrastive causal explanation for graph neural network-based vulnerability detection," in Proc. ACM CCS, 2024..

[15] M. Sutton, A. Greene, and P. Amini,Fuzzing: Brute Force Vulnerability Discovery.Boston, MA, USA: Addison-Wesley Professional, 2007.

[16] H. Riggs et al.,"Impact, vulnerabilities, and mitigation strategies for cyber attacks on critical infrastructure,"Electronics, vol. 12, no. 6, 2023.

[17] K. J. Raval,"A survey on safeguarding critical infrastructures: attacks, AI, and security challenges,"Procedia Computer Science, 2024.

[18] Z. Chu et al., "Counterfactual explanation for graph neural network-based vulnerability detection," in Proc. ACM ISSTA, 2024.

[19] Z. Sheng et al.,"A Survey of Vulnerability Detection Techniques and Insights,"ACM Computing Surveys, 2024.

[20] J. Li, B. Zhao, and C. Zhang, "Fuzzing: A Survey," Cybersecurity, vol. 1, no. 1, 2018.

[21] V. Manès, H. Han, S. Kim, M. Woo, and D. Brumley, "The Art, Science, and Engineering of Fuzzing: A Survey," IEEE Transactions on Software Engineering, vol. 47, no. 11, 2021.

[22] J. Zhang et al.,"Vulnerability Detection by Learning From Syntax-Based and Semantic Code Representations,"IEEE Transactions on Software Engineering, 2023.

[23] Z. Li et al.,"VISION: Vulnerability Identification in Source Code Using Graph-Based Deep Learning,"2023.

[24] Y. Zhang et al., "Revisiting the Generalization Ability of Learning-Based Vulnerability Detection," IEEE Trans. Softw. Eng., 2024.

[25] J. Fan et al., "A Survey of Deep Learning-Based Vulnerability Detection Methods," IEEE Access, vol. 11, pp. 120742–120764, 2023.

[26] J. Chen, X. Xia, D. Lo, J. Grundy, and X. Yang,"Defining Smart Contract Vulnerabilities in Ethereum,"IEEE Transactions on Software Engineering, 2023.

[27] M. Ohm et al., "Backstabber's knife collection: A review of open source software supply chain attacks," in Proc. ACM CCS, 2023.